

Sample 1: CVE-2014-4667--associola.c--sctp_association_free

Vulnerable Code:

```
void sctp_association_free(struct sctp_association *asoc)
{
    struct sock *sk = asoc->base.sk;
    struct sctp_transport *transport;
    struct list_head *pos, *temp;
    int i;

    /* Only real associations count against the endpoint, so
     * don't bother for if this is a temporary association.
     */
    if (!asoc->temp) {
        list_del(&asoc->asocs);

        /* Decrement the backlog value for a TCP-style listening
         * socket.
         */
        if (sctp_style(sk, TCP) && sctp_sstate(sk, LISTENING))
            sk->sk_ack_backlog--;
    }

    /* Mark as dead, so other users can know this structure is
     * going away.
     */
    asoc->base.dead = true;

    /* Dispose of any data lying around in the outqueue. */
    sctp_outq_free(&asoc->outqueue);

    /* Dispose of any pending messages for the upper layer. */
    sctp_ulpq_free(&asoc->ulpq);

    /* Dispose of any pending chunks on the inqueue. */
    sctp_inq_free(&asoc->base.inqueue);

    sctp_tsnmap_free(&asoc->peer.tsn_map);

    /* Free ssmap storage. */
    sctp_ssmap_free(asoc->ssmap);

    /* Clean up the bound address list. */
}
```

```

sctp_bind_addr_free(&asoc->base.bind_addr);

/* Do we need to go through all of our timers and
 * delete them? To be safe we will try to delete all, but we
 * should be able to go through and make a guess based
 * on our state.
 */
for (i = SCTP_EVENT_TIMEOUT_NONE; i < SCTP_NUM_TIMEOUT_TYPES; ++i) {
    if (del_timer(&asoc->timers[i]))
        sctp_association_put(asoc);
}

/* Free peer's cached cookie. */
kfree(asoc->peer.cookie);
kfree(asoc->peer.peer_random);
kfree(asoc->peer.peer_chunks);
kfree(asoc->peer.peer_hmacs);

/* Release the transport structures. */
list_for_each_safe(pos, temp, &asoc->peer.transport_addr_list) {
    transport = list_entry(pos, struct sctp_transport, transports);
    list_del_rcu(pos);
    sctp_transport_free(transport);
}

asoc->peer.transport_count = 0;

sctp_asconf_queue_teardown(asoc);

/* Free pending address space being deleted */
if (asoc->asconf_addr_del_pending != NULL)
    kfree(asoc->asconf_addr_del_pending);

/* AUTH - Free the endpoint shared keys */
sctp_auth_destroy_keys(&asoc->endpoint_shared_keys);

/* AUTH - Free the association shared key */
sctp_auth_key_put(asoc->asoc_shared_key);

sctp_association_put(asoc);
}

```

Ground Truth:

```
- if (!asoc->temp) {
+ if (!list_empty(&asoc->asocs)) {
```

Generated Patch:

1) KitPatch

```
- if (!asoc->temp) {
+ if (!list_empty(&asoc->asocs)) {
```

2) DeepSeek-V3

```
- struct sock *sk = asoc->base.sk;
+ struct sock *sk;
+ sk = asoc->base.sk;
```

3) GPT-5

```
+ struct sctp_endpoint *ep;
+     ep = asoc->ep;
+ asoc->base.dead = true;
+ spin_lock_bh(&asoc->base.sk->sk_lock.slock);
+ asoc->base.sk = NULL;
+ spin_unlock_bh(&asoc->base.sk->sk_lock.slock);
+ asoc->ssnmap = NULL;
+ asoc->asconf_addr_del_pending = NULL;
+ asoc->asoc_shared_key = NULL;
+ asoc->endpoint_shared_keys.keys = NULL;
+ return;
```

4) Claude-Sonnet-4.5

```
+ int timer_deleted = 0;
-     if (del_timer(&asoc->timers[i]))
-         sctp_association_put(asoc);
+     if (del_timer(&asoc->timers[i]))
+         timer_deleted++;
+ for (i = 0; i < timer_deleted; i++)
+     sctp_association_put(asoc);
```

Sample 2: CVE-2012-5532--hv_kvp_daemon.c--main

Vulnerable Code:

```
int main(void)
{
```

```

int fd, len, sock_opt;
int error;
struct cn_msg *message;
struct pollfd pfd;
struct nlmsgghdr *incoming_msg;
struct cn_msg *incoming_cn_msg;
struct hv_kvp_msg *hv_msg;
char *p;
char *key_value;
char *key_name;
int op;
int pool;
char *if_name;
struct hv_kvp_ipaddr_value *kvp_ip_val;

daemon(1, 0);
openlog("KVP", 0, LOG_USER);
syslog(LOG_INFO, "KVP starting; pid is:%d", getpid());
/*
 * Retrieve OS release information.
 */
kvp_get_os_info();

if (kvp_file_init()) {
    syslog(LOG_ERR, "Failed to initialize the pools");
    exit(EXIT_FAILURE);
}

fd = socket(AF_NETLINK, SOCK_DGRAM, NETLINK_CONNECTOR);
if (fd < 0) {
    syslog(LOG_ERR, "netlink socket creation failed; error:%d", fd);
    exit(EXIT_FAILURE);
}
addr.nl_family = AF_NETLINK;
addr.nl_pad = 0;
addr.nl_pid = 0;
addr.nl_groups = CN_KVP_IDX;

error = bind(fd, (struct sockaddr *)&addr, sizeof(addr));
if (error < 0) {
    syslog(LOG_ERR, "bind failed; error:%d", error);
    close(fd);
    exit(EXIT_FAILURE);
}

```

```

}
sock_opt = addr.nl_groups;
setsockopt(fd, 270, 1, &sock_opt, sizeof(sock_opt));
/*
 * Register ourselves with the kernel.
 */
message = (struct cn_msg *)kvp_send_buffer;
message->id.idx = CN_KVP_IDX;
message->id.val = CN_KVP_VAL;

hv_msg = (struct hv_kvp_msg *)message->data;
hv_msg->kvp_hdr.operation = KVP_OP_REGISTER1;
message->ack = 0;
message->len = sizeof(struct hv_kvp_msg);

len = netlink_send(fd, message);
if (len < 0) {
    syslog(LOG_ERR, "netlink_send failed; error:%d", len);
    close(fd);
    exit(EXIT_FAILURE);
}

pfd.fd = fd;

while (1) {
    struct sockaddr *addr_p = (struct sockaddr *) &addr;
    socklen_t addr_l = sizeof(addr);
    pfd.events = POLLIN;
    pfd.revents = 0;
    poll(&pfd, 1, -1);

    len = recvfrom(fd, kvp_rcv_buffer, sizeof(kvp_rcv_buffer), 0,
        addr_p, &addr_l);

    if (len < 0 || addr.nl_pid) {
        syslog(LOG_ERR, "recvfrom failed; pid:%u error:%d %s",
            addr.nl_pid, errno, strerror(errno));
        close(fd);
        return -1;
    }

    incoming_msg = (struct nlmsg_hdr *)kvp_rcv_buffer;
    incoming_cn_msg = (struct cn_msg *)NLMSG_DATA(incoming_msg);
    hv_msg = (struct hv_kvp_msg *)incoming_cn_msg->data;

```

```

/*
 * We will use the KVP header information to pass back
 * the error from this daemon. So, first copy the state
 * and set the error code to success.
 */
op = hv_msg->kvp_hdr.operation;
pool = hv_msg->kvp_hdr.pool;
hv_msg->error = HV_S_OK;

if ((in_hand_shake) && (op == KVP_OP_REGISTER1)) {
    /*
     * Driver is registering with us; stash away the version
     * information.
     */
    in_hand_shake = 0;
    p = (char *)hv_msg->body.kvp_register.version;
    lic_version = malloc(strlen(p) + 1);
    if (lic_version) {
        strcpy(lic_version, p);
        syslog(LOG_INFO, "KVP LIC Version: %s",
            lic_version);
    } else {
        syslog(LOG_ERR, "malloc failed");
    }
    continue;
}

switch (op) {
case KVP_OP_GET_IP_INFO:
    kvp_ip_val = &hv_msg->body.kvp_ip_val;
    if_name =
        kvp_mac_to_if_name((char *)kvp_ip_val->adapter_id);

    if (if_name == NULL) {
        /*
         * We could not map the mac address to an
         * interface name; return error.
         */
        hv_msg->error = HV_E_FAIL;
        break;
    }
    error = kvp_get_ip_info(
        0, if_name, KVP_OP_GET_IP_INFO,

```

```

        kvp_ip_val,
        (MAX_IP_ADDR_SIZE * 2));

    if (error)
        hv_msg->error = error;

    free(if_name);
    break;

case KVP_OP_SET_IP_INFO:
    kvp_ip_val = &hv_msg->body.kvp_ip_val;
    if_name = kvp_get_if_name(
        (char *)kvp_ip_val->adapter_id);
    if (if_name == NULL) {
        /*
         * We could not map the guid to an
         * interface name; return error.
         */
        hv_msg->error = HV_GUID_NOTFOUND;
        break;
    }
    error = kvp_set_ip_info(if_name, kvp_ip_val);
    if (error)
        hv_msg->error = error;

    free(if_name);
    break;

case KVP_OP_SET:
    if (kvp_key_add_or_modify(pool,
        hv_msg->body.kvp_set.data.key,
        hv_msg->body.kvp_set.data.key_size,
        hv_msg->body.kvp_set.data.value,
        hv_msg->body.kvp_set.data.value_size))
        hv_msg->error = HV_S_CONT;

    break;

case KVP_OP_GET:
    if (kvp_get_value(pool,
        hv_msg->body.kvp_set.data.key,
        hv_msg->body.kvp_set.data.key_size,
        hv_msg->body.kvp_set.data.value,
        hv_msg->body.kvp_set.data.value_size))
        hv_msg->error = HV_S_CONT;

```

```

        break;

case KVP_OP_DELETE:
    if (kvp_key_delete(pool,
        hv_msg->body.kvp_delete.key,
        hv_msg->body.kvp_delete.key_size))
        hv_msg->error = HV_S_CONT;
    break;

default:
    break;
}

if (op != KVP_OP_ENUMERATE)
    goto kvp_done;

/*
 * If the pool is KVP_POOL_AUTO, dynamically generate
 * both the key and the value; if not read from the
 * appropriate pool.
 */
if (pool != KVP_POOL_AUTO) {
    if (kvp_pool_enumerate(pool,
        hv_msg->body.kvp_enum_data.index,
        hv_msg->body.kvp_enum_data.data.key,
        HV_KVP_EXCHANGE_MAX_KEY_SIZE,
        hv_msg->body.kvp_enum_data.data.value,
        HV_KVP_EXCHANGE_MAX_VALUE_SIZE))
        hv_msg->error = HV_S_CONT;
    goto kvp_done;
}

hv_msg = (struct hv_kvp_msg *)incoming_cn_msg->data;
key_name = (char *)hv_msg->body.kvp_enum_data.data.key;
key_value = (char *)hv_msg->body.kvp_enum_data.data.value;

switch (hv_msg->body.kvp_enum_data.index) {
case FullyQualifiedDomainName:
    kvp_get_domain_name(key_value,
        HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
    strcpy(key_name, "FullyQualifiedDomainName");
    break;
case IntegrationServicesVersion:
    strcpy(key_name, "IntegrationServicesVersion");

```



```

        strcpy(key_value, lic_version);
        break;
case NetworkAddressIPv4:
    kvp_get_ip_info(AF_INET, NULL, KVP_OP_ENUMERATE,
        key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
    strcpy(key_name, "NetworkAddressIPv4");
    break;
case NetworkAddressIPv6:
    kvp_get_ip_info(AF_INET6, NULL, KVP_OP_ENUMERATE,
        key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
    strcpy(key_name, "NetworkAddressIPv6");
    break;
case OSBuildNumber:
    strcpy(key_value, os_build);
    strcpy(key_name, "OSBuildNumber");
    break;
case OSName:
    strcpy(key_value, os_name);
    strcpy(key_name, "OSName");
    break;
case OSMajorVersion:
    strcpy(key_value, os_major);
    strcpy(key_name, "OSMajorVersion");
    break;
case OSMinorVersion:
    strcpy(key_value, os_minor);
    strcpy(key_name, "OSMinorVersion");
    break;
case OSVersion:
    strcpy(key_value, os_version);
    strcpy(key_name, "OSVersion");
    break;
case ProcessorArchitecture:
    strcpy(key_value, processor_arch);
    strcpy(key_name, "ProcessorArchitecture");
    break;
default:
    hv_msg->error = HV_S_CONT;
    break;
}
/*
* Send the value back to the kernel. The response is
* already in the receive buffer. Update the cn_msg header to
* reflect the key value that has been added to the message

```

```

        */
kvp_done:

    incoming_cn_msg->id.idx = CN_KVP_IDX;
    incoming_cn_msg->id.val = CN_KVP_VAL;
    incoming_cn_msg->ack = 0;
    incoming_cn_msg->len = sizeof(struct hv_kvp_msg);

    len = netlink_send(fd, incoming_cn_msg);
    if (len < 0) {
        syslog(LOG_ERR, "net_link send failed; error:%d", len);
        exit(EXIT_FAILURE);
    }
}
}

```

Ground Truth:

```

-   if (len < 0 || addr.nl_pid) {
+   if (len < 0) {
+   if (addr.nl_pid) {
+       syslog(LOG_WARNING, "\"Received packet from untrusted pid:%u\"",
+           addr.nl_pid);
+       continue;
+   }

```

Generated Patch:

1) KitPatch

```

-   if (len < 0 || addr.nl_pid) {
+   if (len < 0) {
+   if (addr.nl_pid) {
+       syslog(LOG_WARNING, "\"Received packet from unsupported pid:%u\"",
+           addr.nl_pid);
+       continue;
+   }

```

2) DeepSeek-V3

```

-   (MAX_IP_ADDR_SIZE * 2));
+   sizeof(struct hv_kvp_ipaddr_value));

```

3) GPT-5

```

+   size_t n;
-       strcpy(key_name, "IntegrationServicesVersion");
-       strcpy(key_value, lic_version);

```

```

+     snprintf(key_name,
+         HV_KVP_EXCHANGE_MAX_KEY_SIZE,
+         "IntegrationServicesVersion");
+     if (lic_version)
+         snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", lic_version);
+     else
+         hv_msg->error = HV_S_CONT; /* no version available */
-         key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
-     strcpy(key_name, "NetworkAddressIPv4");
+         key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "NetworkAddressIPv4");
-         key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
-     strcpy(key_name, "NetworkAddressIPv6");
+         key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "NetworkAddressIPv6");
-     strcpy(key_value, os_build);
-     strcpy(key_name, "OSBuildNumber");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", os_build);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "OSBuildNumber");
-     strcpy(key_value, os_name);
-     strcpy(key_name, "OSName");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", os_name);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "OSName");
-     strcpy(key_value, os_major);
-     strcpy(key_name, "OSMajorVersion");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", os_major);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "OSMajorVersion");
-     strcpy(key_value, os_minor);
-     strcpy(key_name, "OSMinorVersion");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", os_minor);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "OSMinorVersion");
-     strcpy(key_value, os_version);
-     strcpy(key_name, "OSVersion");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", os_version);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "OSVersion");
-     strcpy(key_value, processor_arch);
-     strcpy(key_name, "ProcessorArchitecture");
+     snprintf(key_value, HV_KVP_EXCHANGE_MAX_VALUE_SIZE, "%s", processor_arch);
+     snprintf(key_name, HV_KVP_EXCHANGE_MAX_KEY_SIZE, "ProcessorArchitecture");

```

4) Claude-Sonnet-4.5

```

+     if (len < sizeof(struct nlmsg_hdr) + sizeof(struct cn_msg)) {
+         syslog(LOG_ERR, "received message too small: %d bytes", len);
+         continue;

```

```

+     }
+     if (len < sizeof(struct nlmsg_hdr) + sizeof(struct cn_msg) +
+         sizeof(struct hv_kvp_msg)) {
+         syslog(LOG_ERR, "received message missing hv_kvp_msg data");
+         continue;
+     }
+
+     if (incoming_cn_msg->len > sizeof(struct hv_kvp_msg))
+         incoming_cn_msg->len = sizeof(struct hv_kvp_msg);
+     if (lic_version) {
+         if (lic_version && strlen(p) < 256) {
+             strcpy(key_name, "FullyQualifiedDomainName");
+             strncpy(key_name, "FullyQualifiedDomainName",
+                 HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1);
+             key_name[HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1] = '\0';
+             strcpy(key_name, "IntegrationServicesVersion");
+             strcpy(key_value, lic_version);
+             strncpy(key_name, "IntegrationServicesVersion",
+                 HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1);
+             key_name[HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1] = '\0';
+             strncpy(key_value, lic_version ? lic_version : "",
+                 HV_KVP_EXCHANGE_MAX_VALUE_SIZE - 1);
+             key_value[HV_KVP_EXCHANGE_MAX_VALUE_SIZE - 1] = '\0';
+             strcpy(key_name, "NetworkAddressIPv6");
+             strncpy(key_name, "NetworkAddressIPv6",
+                 HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1);
+             key_name[HV_KVP_EXCHANGE_MAX_KEY_SIZE - 1] = '\0';
+         case OSBuildNumber:
+             strcpy(key_value, os_build);
+             strcpy(key_name, "OSBuildNumber");
+             break;
+         case OSName:
+             strcpy(key_value, os_name);
+             strcpy(key_name, "OSName");
+             break;
+         case OSMajorVersion:
+             strcpy(key_value, os_major);
+             strcpy(key_name, "OSMajorVersion");
+             break;

```

Sample 3: CVE-2011-1585--connect.c--cifs_get_smb_ses

Vulnerable Code:

```
cifs_get_smb_ses(struct TCP_Server_Info *server, struct smb_vol *volume_info)
{
    int rc = -ENOMEM, xid;
    struct cifsSesInfo *ses;

    xid = GetXid();

    ses = cifs_find_smb_ses(server, volume_info->username);
    if (ses) {
        cFYI(1, "Existing smb sess found (status=%d)", ses->status);

        /* existing SMB ses has a server reference already */
        cifs_put_tcp_session(server);

        mutex_lock(&ses->session_mutex);
        rc = cifs_negotiate_protocol(xid, ses);
        if (rc) {
            mutex_unlock(&ses->session_mutex);
            /* problem -- put our ses reference */
            cifs_put_smb_ses(ses);
            FreeXid(xid);
            return ERR_PTR(rc);
        }
        if (ses->need_reconnect) {
            cFYI(1, "Session needs reconnect");
            rc = cifs_setup_session(xid, ses,
                                   volume_info->local_nls);
            if (rc) {
                mutex_unlock(&ses->session_mutex);
                /* problem -- put our reference */
                cifs_put_smb_ses(ses);
                FreeXid(xid);
                return ERR_PTR(rc);
            }
        }
        mutex_unlock(&ses->session_mutex);
        FreeXid(xid);
        return ses;
    }

    cFYI(1, "Existing smb sess not found");
```

```

ses = sesInfoAlloc();
if (ses == NULL)
    goto get_ses_fail;

/* new SMB session uses our server ref */
ses->server = server;
if (server->addr.sockAddr6.sin6_family == AF_INET6)
    sprintf(ses->serverName, "%pI6",
            &server->addr.sockAddr6.sin6_addr);
else
    sprintf(ses->serverName, "%pI4",
            &server->addr.sockAddr.sin_addr.s_addr);

if (volume_info->username)
    strncpy(ses->userName, volume_info->username,
            MAX_USERNAME_SIZE);

/* volume_info->password freed at unmount */
if (volume_info->password) {
    ses->password = kstrdup(volume_info->password, GFP_KERNEL);
    if (!ses->password)
        goto get_ses_fail;
}
if (volume_info->domainname) {
    int len = strlen(volume_info->domainname);
    ses->domainName = kmalloc(len + 1, GFP_KERNEL);
    if (ses->domainName)
        strcpy(ses->domainName, volume_info->domainname);
}
ses->linux_uid = volume_info->linux_uid;
ses->overrideSecFlg = volume_info->secFlg;

mutex_lock(&ses->session_mutex);
rc = cifs_negotiate_protocol(xid, ses);
if (!rc)
    rc = cifs_setup_session(xid, ses, volume_info->local_nls);
mutex_unlock(&ses->session_mutex);
if (rc)
    goto get_ses_fail;

/* success, put it on the list */
write_lock(&cifs_tcp_ses_lock);
list_add(&ses->smb_ses_list, &server->smb_ses_list);
write_unlock(&cifs_tcp_ses_lock);

```

```

        FreeXid(xid);
        return ses;

get_ses_fail:
    sesInfoFree(ses);
    FreeXid(xid);
    return ERR_PTR(rc);
}

```

Ground Truth:

```

- ses = cifs_find_smb_ses(server, volume_info->username);
+ ses = cifs_find_smb_ses(server, volume_info);

```

Generated Patch:

1) KitPatch

```

- ses = cifs_find_smb_ses(server, volume_info->username);
+ ses = cifs_find_smb_ses(server, volume_info);

```

2) DeepSeek-V3

```

+ read_lock(&cifs_tcp_ses_lock);
+ read_unlock(&cifs_tcp_ses_lock);
+ mutex_lock(&ses->session_mutex);
+ mutex_unlock(&ses->session_mutex);

```

3) GPT-5

```

- int rc = -ENOMEM, xid;
+ int rc = -ENOMEM, xid;
- if (server->addr.sockAddr6.sin6_family == AF_INET6)
-     sprintf(ses->serverName, "%pI6",
-             &server->addr.sockAddr6.sin6_addr);
- else
-     sprintf(ses->serverName, "%pI4",
-             &server->addr.sockAddr.sin_addr.s_addr);
+ if (server->addr.sockAddr6.sin6_family == AF_INET6)
+     snprintf(ses->serverName, sizeof(ses->serverName), "%pI6",
+             &server->addr.sockAddr6.sin6_addr);
+ else
+     snprintf(ses->serverName, sizeof(ses->serverName), "%pI4",
+             &server->addr.sockAddr.sin_addr.s_addr);
- strncpy(ses->userName, volume_info->username,
-         MAX_USERNAME_SIZE);
+ strncpy(ses->userName, volume_info->username,
+         sizeof(ses->userName));

```

```

+   else
+       ses->userName[0] = '\0';
-       int len = strlen(volume_info->domainname);
-       ses->domainName = kmalloc(len + 1, GFP_KERNEL);
-       if (ses->domainName)
-           strcpy(ses->domainName, volume_info->domainname);
+       ses->domainName = kstrdup(volume_info->domainname, GFP_KERNEL);
+       if (!ses->domainName) {
+           rc = -ENOMEM;
+           goto get_ses_fail;
+       }
-       sesInfoFree(ses);
+   if (ses)
+       sesInfoFree(ses);

```

4) Claude-Sonnet-4.5

```

+   struct TCP_Server_Info *tcp_ses;
-       cifs_put_tcp_session(server);
+       cifs_put_tcp_session(server);
-       strncpy(ses->userName, volume_info->username,
-               MAX_USERNAME_SIZE);
+       strncpy(ses->userName, volume_info->username,
+               MAX_USERNAME_SIZE - 1);
+       ses->userName[MAX_USERNAME_SIZE - 1] = '\0';

```

Sample 4: CVE-2014-3538--apprentice.c--string_modifier_check

Vulnerable Code:

```

string_modifier_check(struct magic_set *ms, struct magic *m)
{
    if ((ms->flags & MAGIC_CHECK) == 0)
        return 0;

    if (m->type != FILE_PSTRING && (m->str_flags & PSTRING_LEN) != 0) {
        file_magwarn(ms,
            "'/BHhLL' modifiers are only allowed for pascal strings\n");
        return -1;
    }
    switch (m->type) {
    case FILE_BESTRING16:
    case FILE_LESTRING16:

```



```

        if (m->str_flags != 0) {
            file_magwarn(ms,
                "no modifiers allowed for 16-bit strings\n");
            return -1;
        }
        break;
case FILE_STRING:
case FILE_PSTRING:
    if ((m->str_flags & REGEX_OFFSET_START) != 0) {
        file_magwarn(ms,
            "'/%c' only allowed on regex and search\n",
            CHAR_REGEX_OFFSET_START);
        return -1;
    }
    break;
case FILE_SEARCH:
    if (m->str_range == 0) {
        file_magwarn(ms,
            "missing range; defaulting to %d\n",
            STRING_DEFAULT_RANGE);
        m->str_range = STRING_DEFAULT_RANGE;
        return -1;
    }
    break;
case FILE_REGEX:
    if ((m->str_flags & STRING_COMPACT_WHITESPACE) != 0) {
        file_magwarn(ms, "'/%c' not allowed on regex\n",
            CHAR_COMPACT_WHITESPACE);
        return -1;
    }
    if ((m->str_flags & STRING_COMPACT_OPTIONAL_WHITESPACE) != 0) {
        file_magwarn(ms, "'/%c' not allowed on regex\n",
            CHAR_COMPACT_OPTIONAL_WHITESPACE);
        return -1;
    }
    break;
default:
    file_magwarn(ms, "coding error: m->type=%d\n",
        m->type);
    return -1;
}
return 0;
}

```

Ground Truth:

```
- if (m->type != FILE_PSTRING && (m->str_flags & PSTRING_LEN) != 0) {  
+ if ((m->type != FILE_REGEX || (m->str_flags & REGEX_LINE_COUNT) == 0) &&  
+ (m->type != FILE_PSTRING && (m->str_flags & PSTRING_LEN) != 0)) {
```

Generated Patch:**1) KitPatch**

```
- ses = cifs_find_smb_ses(server, volume_info->username);  
+ ses = cifs_find_smb_ses(server, volume_info);
```

2) DeepSeek-V3

```
- if (m->type != FILE_PSTRING && (m->str_flags & PSTRING_LEN) != 0)  
+ if ((m->type != FILE_REGEX || (m->str_flags & REGEX_LINE_COUNT) == 0) &&  
+ (m->type != FILE_PSTRING && (m->str_flags & PSTRING_LEN) != 0))
```

3) GPT-5

```
- if ((ms->flags & MAGIC_CHECK) == 0)  
-     return 0;  
+ if (!ms || !m)  
+     return -1;  
+ if ((ms->flags & MAGIC_CHECK) == 0)  
+     return 0;  
-         "missing range; defaulting to %d\n",  
-         STRING_DEFAULT_RANGE);  
-     m->str_range = STRING_DEFAULT_RANGE;  
-     return -1;  
+         "missing range; defaulting to %d\n",  
+         STRING_DEFAULT_RANGE);  
+ if (STRING_DEFAULT_RANGE > MAX_STRING_RANGE)  
+     m->str_range = MAX_STRING_RANGE;  
+ else  
+     m->str_range = STRING_DEFAULT_RANGE;  
+     return 0;  
-     return -1;  
+     return -1;  
+ if (m->str_flags & ~ALL_STRING_FLAGS) {  
+     file_magwarn(ms, "invalid string flag combination: 0x%x\n",  
+         m->str_flags);  
+     return -1;  
+ }
```

4) Claude-Sonnet-4.5

```
-     return -1;
```

Sample 5: CVE-2016-2544--seq_queue.c--queue_delete

Vulnerable Code:

```
static void queue_delete(struct snd_seq_queue *q)
{
    /* stop and release the timer */
    snd_seq_timer_stop(q->timer);
    snd_seq_timer_close(q);
    /* wait until access free */
    snd_use_lock_sync(&q->use_lock);
    /* release resources... */
    snd_seq_prioq_delete(&q->tickq);
    snd_seq_prioq_delete(&q->timeq);
    snd_seq_timer_delete(&q->timer);

    kfree(q);
}
```

Ground Truth:

```
+ mutex_lock(&q->timer_mutex);
+ mutex_unlock(&q->timer_mutex);
```

Generated Patch:

1) KitPatch

```
+ mutex_lock(&q->timer_mutex);
+ mutex_unlock(&q->timer_mutex);
```

2) DeepSeek-V3

```
- snd_use_lock_sync(&q->use_lock);
+ snd_use_lock_sync(&q->use_lock);
```

3) GPT-5

```
- snd_seq_timer_stop(q->timer);
- snd_seq_timer_close(q);
- snd_use_lock_sync(&q->use_lock);
- snd_seq_prioq_delete(&q->tickq);
- snd_seq_prioq_delete(&q->timeq);
- snd_seq_timer_delete(&q->timer);
- kfree(q);
+ if (!q)
+     return;
+
```

```

+   if (q->timer) {
+       snd_seq_timer_stop(q->timer);
+       snd_seq_timer_close(q);
+   }
+   snd_use_lock_sync(&q->use_lock);
+   if (!list_empty(&q->tickq.list))
+       snd_seq_prioq_delete(&q->tickq);
+   if (!list_empty(&q->timeq.list))
+       snd_seq_prioq_delete(&q->timeq);
+   if (q->timer) {
+       snd_seq_timer_delete(&q->timer);
+       q->timer = NULL;
+   }
+   smp_wmb();
+   kfree(q);

```

4) Claude-Sonnet-4.5

```

+   queuefree(q);

```